



Research Intake 2026

Day 8

Training Deep Networks

A Beginner's Guide for Students

Understanding AI, Machine Learning, and Deep Learning

Gudsky Research Foundation

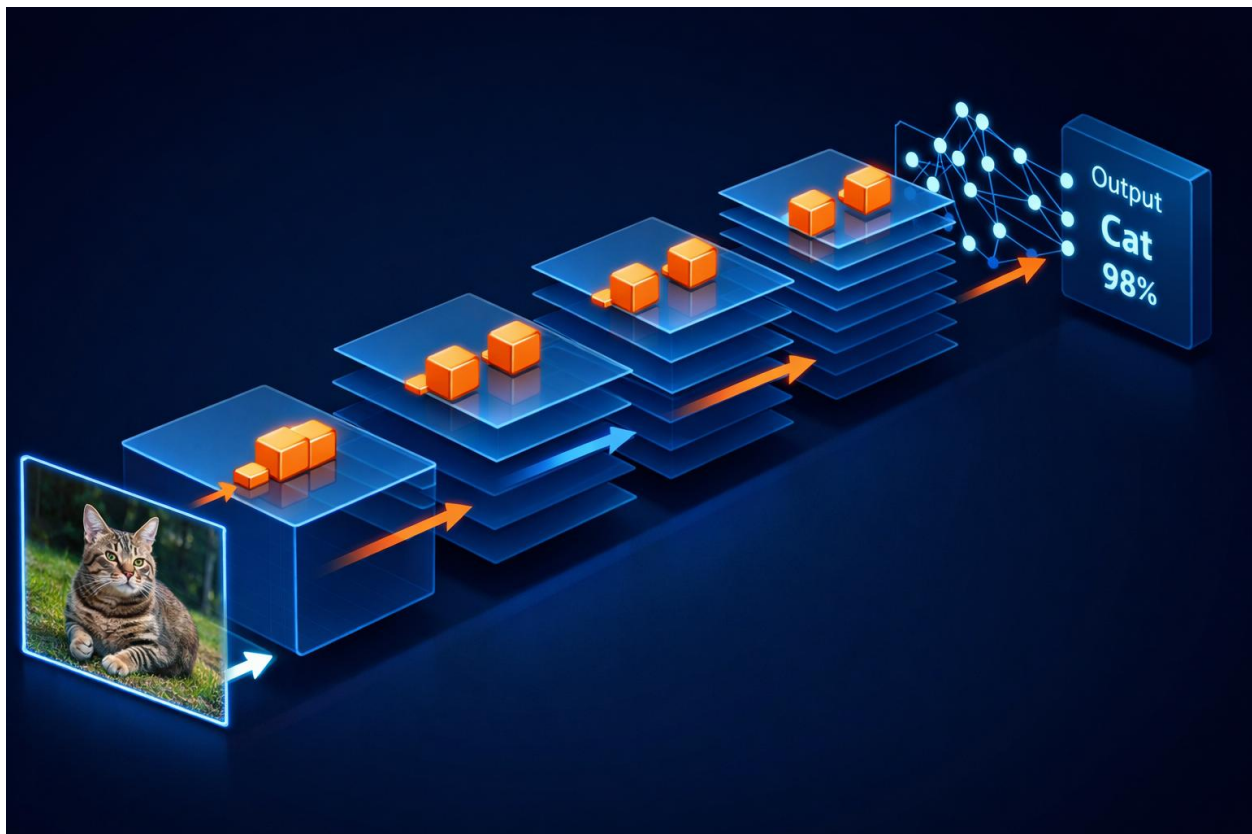
Table of Contents

1. The Training Loop in Depth
2. Overfitting & Regularisation
3. Optimisation Algorithms
4. Weight Initialisation
5. Batch Normalisation
6. Learning Rate Scheduling
7. Convolutional Neural Networks (CNN)
8. Object Detection — R-CNN Family
9. Recurrent Neural Networks (RNN)
10. Hands-On Lab — CIFAR-10 CNN with PyTorch
11. Glossary & Review Questions

Learning Objectives

By the end of this session, students will be able to:

- Distinguish training, validation, and test sets and explain why all three splits are essential.
- Identify overfitting from a loss curve and apply L1/L2 regularisation, Dropout, and Early Stopping correctly.
- Explain the Bias-Variance tradeoff and its practical implications for model selection.
- Compare SGD, SGD with Momentum, RMSProp, Adam, and AdamW — and choose the right optimiser for a given task.
- Explain the architecture of Convolutional Neural Networks (CNNs) including convolution, pooling, and fully-connected layers.
- Understand the R-CNN family (R-CNN, Fast R-CNN, Faster R-CNN, Mask R-CNN) for object detection.
- Understand Recurrent Neural Networks (RNNs), LSTMs, and GRUs for sequential data.
- Explain why zero initialisation fails and apply Xavier/He initialisation correctly.
- Describe Internal Covariate Shift and explain how Batch Normalisation solves it.
- Implement Step Decay, Cosine Annealing, Warmup, and Cyclical LR schedules in PyTorch.
- Write effective image generation prompts for diffusion models and understand prompt engineering principles.
- Train a CNN on CIFAR-10 applying all Day 8 techniques: Adam, Dropout, BatchNorm, and LR scheduling.



01 The Training Loop in Depth

Day 7 introduced the basic concept of a training loop. Day 8 begins by clarifying the precise vocabulary and explaining how data is partitioned — choices that have a massive impact on your ability to evaluate and trust a model.

1.1 Epochs, Batches, and Iterations

Three terms are frequently confused. Understanding their exact relationship lets you interpret training logs and set hyperparameters correctly.

Term	Definition	Example (60,000 samples, batch=200)	Typical Range
Epoch	One complete pass through the entire training dataset.	1 epoch = all 60,000 samples seen once	10–200 epochs
Batch (Mini-batch)	A small subset of training samples processed together before a weight update.	Each batch = 200 samples	32–512 samples
Iteration (Step)	One forward pass + one backward pass + one weight update on one batch.	1 epoch = $60,000 / 200 = 300$ iterations	Derived from above

💡 Why Mini-Batches?

Using the entire dataset for one update (Batch GD) is computationally expensive and gets stuck in sharp minima.

Using a single sample (SGD) is noisy and slow to converge.

Mini-batches offer the best of both worlds:

- Efficient use of vectorised GPU operations.
- Noisy enough to escape sharp minima (acts as regulariser).
- Smooth enough to converge reliably.

Typical batch sizes: 32, 64, 128, 256. Larger batches → less noise but risk sharper minima and need more memory.

1.2 The Three Dataset Splits

A fundamental rule: the test set must never influence training or model selection decisions. This requires a strict three-way split.

Split	Typical Size	Purpose	Touched During Training?
Training Set	60–80%	Model learns weights via backpropagation.	Yes — every iteration
Validation Set	10–20%	Tune hyperparameters, compare architectures, trigger early stopping.	Yes — after each epoch (no gradients)

Test Set	10–20%	Final unbiased evaluation reported in papers and products.	No — only once at the end
----------	--------	--	---------------------------

⚠ Data Leakage — The Most Common Mistake

If you use the test set to make any decision — including choosing the best epoch, selecting hyperparameters, or deciding to add more layers — you have contaminated it.

Your reported performance will be optimistic and will not generalise.

RULE: Treat the test set as a sealed envelope. Open it once.

1.3 K-Fold Cross-Validation

When your dataset is small, a single train/val split may be unreliable (high variance depending on which samples end up in each split). K-Fold Cross-Validation addresses this:

🔍 K-Fold Cross-Validation

When your dataset is small, a single train/val split may be unreliable — high variance depending on which samples end up in each split.

STEP 1: Divide the training data into K equal folds (typically $K = 5$ or 10).

STEP 2: Train K separate models — each time, use K-1 folds for training and 1 fold for validation.

STEP 3: Average validation metrics across all K runs to get a stable estimate.

Downside: K times more expensive to train. Not always practical for deep learning, but valuable for smaller datasets or hyperparameter sweeps.

02 Overfitting & Regularisation

Overfitting is the central challenge of supervised learning. A model that overfits memorises the training data — including its noise — rather than learning the underlying pattern. It performs well on training data but fails on unseen examples.

2.1 Recognising Overfitting — Loss Curve Analysis

Phase	Training Loss	Validation Loss	Diagnosis
Early training	High → Decreasing	High → Decreasing	Underfitting — model is still learning
Sweet spot	Low and decreasing	Low and decreasing	Good fit — model is generalising well
Overfitting zone	Still decreasing	Increasing or plateauing	Overfitting — model is memorising training data

Visual Rule of Thumb

If validation loss diverges upward from training loss, stop training.

The gap between training and validation loss is called the generalisation gap — minimising it is the goal of regularisation.

Plot training and validation curves after EVERY run. Don't fly blind.

2.2 The Bias-Variance Tradeoff

Every model's total error decomposes into three fundamental components that cannot all be reduced simultaneously:

Error Decomposition

Total Error = Bias² + Variance + Irreducible Noise

Bias: Error from wrong assumptions. A high-bias model is too simple (underfits).

Example: fitting a straight line to quadratic data.

Variance: Error from sensitivity to fluctuations in training data. High-variance models overfit.

Example: a 100-layer network on 100 training samples.

Irreducible: Inherent randomness in the data. Cannot be reduced by any model.

Scenario	Bias	Variance	Solution
Underfitting	High	Low	Increase model capacity, train longer, reduce regularisation
Good fit	Low	Low	You're done — deploy!

Overfitting	Low	High	Regularise, get more data, reduce model complexity
-------------	-----	------	--

2.3 L1 and L2 Weight Regularisation

Regularisation adds a penalty term to the loss function that discourages large weights, preventing the model from relying too heavily on any single feature.

Regularised Loss Functions

L2 (Ridge): $L_{\text{total}} = L_{\text{task}} + \lambda \cdot \sum w_i^2$

L1 (Lasso): $L_{\text{total}} = L_{\text{task}} + \lambda \cdot \sum |w_i|$

λ (lambda) is the regularisation strength hyperparameter.

Higher λ = stronger penalty = simpler model.

L2 (Ridge / Weight Decay)	L1 (Lasso)
Pushes all weights towards zero (but rarely exactly zero)	Pushes many weights to exactly zero (sparse solution)
$2\lambda w_i$ (proportional to weight magnitude)	$\lambda \cdot \text{sign}(w_i)$ (constant magnitude)
No — keeps all features	Yes — performs implicit feature selection
Yes — via <code>weight_decay</code> in optimiser	Less common; used mainly in linear models

2.4 Dropout

Dropout is one of the most impactful regularisation techniques ever invented. During each training forward pass, each neuron is independently set to zero with probability p (the dropout rate), effectively training a different sub-network each time.

How Dropout Works

TRAINING: Each neuron is 'dropped' with probability p . The surviving neurons are scaled up by $1/(1-p)$ to maintain expected output magnitude.

INFERENCE: All neurons are active. No scaling is needed (handled during training via inverted dropout).

EFFECT: Forces the network to learn redundant representations — no single neuron can be relied upon.

INTUITION: Analogous to ensemble learning — you effectively train and average thousands of sub-networks.

TYPICAL VALUES: 0.2–0.5 for dense layers. Avoid dropout on the final output layer.

```
import torch.nn as nn
```

```
class RegularisedMLP(nn.Module):
    def __init__(self):
```

```

super().__init__()
self.net = nn.Sequential(
    nn.Linear(784, 512),
    nn.ReLU(),
    nn.Dropout(p=0.4),    # Drop 40% of neurons during training
    nn.Linear(512, 256),
    nn.ReLU(),
    nn.Dropout(p=0.3),
    nn.Linear(256, 10)
)

def forward(self, x):
    return self.net(x)

# CRITICAL: Call model.eval() before inference — disables Dropout and uses BN running stats
model.train() # Dropout ACTIVE — use during training
model.eval()  # Dropout DISABLED — use during validation and inference

```

2.5 Early Stopping

Early Stopping monitors validation loss during training and halts when it stops improving — before the model has time to overfit. It is both the simplest and most effective regularisation technique for deep networks.

Parameter	Description
Patience	Number of epochs to wait for improvement before stopping. Typical: 5–20 epochs.
Min delta	Minimum improvement to count as 'progress'. Typical: 0.001–0.01.
Monitor	Metric being watched — usually <code>val_loss</code> or <code>val_accuracy</code> .
Restore best weights	ALWAYS set to True — revert to the checkpoint with best <code>val_loss</code> when stopping.

Common Mistake: Patience Too Low

If patience = 1 or 2, training will stop prematurely during natural oscillations in validation loss. Use a patience of at least 5–10 epochs, especially with noisy datasets or small learning rates. Too-high patience is usually better than too-low — the cost is only extra training time.

03 Optimisation Algorithms

The optimiser is the algorithm that updates weights after each backward pass. Choosing the right optimiser affects convergence speed, final performance, and training stability. Plain SGD is rarely the best choice for deep networks.

3.1 Plain SGD and Its Problems

SGD Update Rule

$$w \leftarrow w - \eta \cdot \nabla L(w)$$

η = learning rate | $\nabla L(w)$ = gradient of loss with respect to weight w

Plain SGD suffers from three fundamental problems:

- Ravines: Steep curvature in one direction causes oscillations while making slow progress.
- Saddle Points: In high-dimensional spaces, saddle points (zero gradient, not a minimum) trap SGD.
- Learning Rate Sensitivity: A single fixed LR must work for every parameter — impossible to optimise well.

3.2 SGD with Momentum

Momentum accumulates a velocity vector in the direction of persistent gradients, accelerating SGD in the right direction and dampening oscillations.

Momentum Update

$$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot \nabla L(w)$$

$$w \leftarrow w - \eta \cdot v_t$$

β (momentum coefficient) ≈ 0.9 is standard.

v_t is the exponential moving average of gradients.

3.3 RMSProp

RMSProp adapts the learning rate per-parameter by dividing by the root mean square of recent gradients. Ideal for recurrent networks and non-stationary objectives.

RMSProp Update

$$s_t = \beta \cdot s_{t-1} + (1 - \beta) \cdot (\nabla L)^2$$

$$w \leftarrow w - (\eta / \sqrt{s_t + \epsilon}) \cdot \nabla L$$

ϵ (epsilon) $\approx 1e-8$ prevents division by zero.

Parameters with large gradients get smaller updates; sparse parameters get larger updates.

3.4 Adam — Adaptive Moment Estimation

Adam combines the benefits of Momentum (first moment) and RMSProp (second moment). It is the de facto standard optimiser for deep learning and works well across a wide range of architectures with minimal tuning.

Adam Update

$$\begin{aligned} m_t &= \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot \nabla L && \leftarrow \text{1st moment (mean of gradients)} \\ v_t &= \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot (\nabla L)^2 && \leftarrow \text{2nd moment (uncentred variance)} \\ \hat{m}_t &= m_t / (1 - \beta_1^t) && \leftarrow \text{bias-corrected 1st moment} \\ \hat{v}_t &= v_t / (1 - \beta_2^t) && \leftarrow \text{bias-corrected 2nd moment} \\ w &\leftarrow w - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) \end{aligned}$$

Defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1e-8$, $\eta = 0.001$

3.5 AdamW — Adam with Decoupled Weight Decay

A key flaw in Adam: L2 regularisation interacts incorrectly with adaptive learning rates, providing unequal regularisation across parameters. AdamW fixes this by decoupling weight decay from the gradient update.

AdamW Update

$$w \leftarrow w - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) - \eta \cdot \lambda \cdot w$$

The second term ($\eta \cdot \lambda \cdot w$) applies weight decay directly to the weights, independently of gradient scaling.

Preferred optimiser for: Transformers (GPT, BERT, ViT), large CNNs.

3.6 Optimiser Comparison Table

Optimiser	Per-Param LR?	Momentum?	Best Use Case
SGD	No	No	Simple tasks with lots of tuning time
SGD + Momentum	No	Yes	CNNs; often best final accuracy with careful tuning
RMSProp	Yes	No	RNNs; non-stationary objectives
Adam	Yes	Yes	Default starting point; transformers; fast convergence

AdamW	Yes	Yes	Transformers; any Adam use case — PREFERRED over Adam
-------	-----	-----	---

💡 Which Optimiser Should I Use?

Start with AdamW (lr=1e-3, weight_decay=0.01) — it works well for 90% of tasks with no tuning.

For very large datasets with tuning time: SGD + Momentum + Cosine Annealing can outperform Adam on CNNs.

For RNNs and LSTMs: Use RMSProp or Adam.

For Transformers (GPT, BERT, ViT): Always use AdamW with warmup scheduling.

04 Weight Initialisation

How weights are set before training begins has a profound effect on whether training succeeds at all. Poor initialisation can cause gradients to vanish or explode within the first few forward passes, making training impossibly slow or unstable.

4.1 Why Zero Initialisation Fails — The Symmetry Problem

⚠ The Symmetry Problem

If all weights in a layer are identical, every neuron computes the EXACT SAME output for any input. Backpropagation computes identical gradients for all neurons → every weight receives the same update. After the update, all weights are still identical — just with a different value.

The network will NEVER break symmetry. All neurons in a layer remain identical forever.

Result: An N-neuron layer is functionally equivalent to a 1-neuron layer. You lose ALL representational capacity.

4.2 Xavier / Glorot Initialisation (for Sigmoid / Tanh)

Xavier initialisation keeps the variance of activations and gradients approximately equal across all layers — preventing both vanishing and exploding signals. Designed for symmetric activation functions centred at zero.

📖 Xavier Uniform Initialisation

$W \sim \text{Uniform}(-a, a)$ where $a = \sqrt{6 / (n_{\text{in}} + n_{\text{out}})}$

Xavier Normal: $W \sim N(0, \sigma^2)$ where $\sigma^2 = 2 / (n_{\text{in}} + n_{\text{out}})$

n_{in} = number of input connections (fan-in)

n_{out} = number of output connections (fan-out)

4.3 He / Kaiming Initialisation (for ReLU Networks)

Xavier was derived assuming symmetric activations. ReLU kills half its inputs (sets negative values to zero), effectively halving the variance at each layer. He initialisation compensates by using a larger variance factor.

📖 He (Kaiming) Initialisation

$W \sim N(0, \sigma^2)$ where $\sigma^2 = 2 / n_{\text{in}}$

He Uniform: $W \sim \text{Uniform}(-a, a)$ where $a = \sqrt{6 / n_{\text{in}}}$

Standard for any network using ReLU or Leaky ReLU activation functions.

Sigmoid / Tanh	Xavier (Glorot): <code>nn.init.xavier_uniform_(layer.weight)</code>
ReLU / Leaky ReLU	He (Kaiming): <code>nn.init.kaiming_uniform_(layer.weight, nonlinearity='relu')</code>
Linear / Output	Xavier or small random: <code>nn.init.xavier_uniform_(layer.weight)</code>
Embeddings	Uniform small range: <code>nn.init.uniform_(embed.weight, -0.1, 0.1)</code>

05 Batch Normalisation

Batch Normalisation (BatchNorm) is one of the most impactful innovations in deep learning history (Ioffe & Szegedy, 2015). It dramatically speeds up training, allows the use of higher learning rates, and reduces sensitivity to weight initialisation.

5.1 Internal Covariate Shift

As weights update during training, the distribution of each layer's inputs changes. A layer trained on one distribution must constantly re-adapt as the previous layer's outputs shift — this is called Internal Covariate Shift.

The Problem — An Analogy

Imagine training a dog to catch a ball — but the ball changes size and colour every throw.

The dog can still learn, but it takes far longer than if the ball were consistent.

Internal Covariate Shift forces each layer to 're-learn' as its inputs keep changing throughout training.

BatchNorm solves this by normalising inputs to each layer, making them consistent regardless of upstream weight changes.

5.2 How Batch Normalisation Works

BatchNorm Forward Pass

Step 1 — Compute batch statistics:

$$\mu_B = (1/m) \cdot \sum x_i \quad \leftarrow \text{batch mean}$$

$$\sigma^2_B = (1/m) \cdot \sum (x_i - \mu_B)^2 \quad \leftarrow \text{batch variance}$$

Step 2 — Normalise:

$$\hat{x}_i = (x_i - \mu_B) / \sqrt{(\sigma^2_B + \epsilon)} \quad \leftarrow \text{zero mean, unit variance}$$

Step 3 — Scale and shift (LEARNABLE):

$$y_i = \gamma \cdot \hat{x}_i + \beta$$

γ and β allow the network to undo normalisation if optimal — learned by backpropagation.

5.3 Training Mode vs Inference Mode

Mode	Behaviour
Training (model.train())	μ and σ computed from current mini-batch. Running averages updated via EMA. γ and β learned via backprop.

Inference (`model.eval()`)

Running averages accumulated during training are used. γ and β fixed (learned values). Single samples work correctly.

⚠ Forgetting `model.eval()` — A Classic Bug

If you forget `model.eval()` before inference, BatchNorm uses mini-batch statistics instead of running averages.

With batch size = 1 (common at inference), batch statistics are extremely noisy.

Predictions will be erratic and performance will collapse.

RULE: `model.train()` during training | `model.eval()` during validation and inference.

💡 Where to Place BatchNorm

Before activation (original paper): Conv → BN → ReLU

After activation (common in practice): Conv → ReLU → BN

The 'before activation' placement is more common in modern architectures (ResNet, EfficientNet).

When in doubt, follow the reference implementation of the architecture you are using.

5.4 Benefits of Batch Normalisation

- Faster convergence: normalised inputs allow higher learning rates without instability.
- Reduces sensitivity to initialisation: even suboptimal initialisation is corrected by normalisation within the first layers.
- Mild regularisation effect: the noise from batch statistics acts similarly to Dropout — often allowing reduction of other regularisation.
- Enables training of much deeper networks: without BatchNorm, training networks deeper than ~10 layers is extremely difficult.

06 Learning Rate Scheduling

A fixed learning rate throughout training is almost always suboptimal. Early in training, a large LR allows fast progress; later, a smaller LR enables fine-grained convergence without overshooting minima.

6.1 Step Decay

The simplest scheduler: reduce the learning rate by a fixed factor (gamma) every N epochs. Easy to understand but requires manual tuning of the step size.

Step Decay Formula

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor \text{epoch} / \text{step_size} \rfloor}$$

Typical: step_size = 10–30 epochs, $\gamma = 0.1$ (reduces LR by 10x every step).

```
from torch.optim.lr_scheduler import StepLR

scheduler = StepLR(optimizer, step_size=30, gamma=0.1)

for epoch in range(num_epochs):
    train_one_epoch(model, dataloader, optimizer)
    scheduler.step() # Must be called AFTER optimizer.step()
```

6.2 Cosine Annealing

Smoothly decays the learning rate following a cosine curve from η_{max} down to η_{min} over T_{max} epochs. Produces better final accuracy than step decay in most settings because the smooth decay avoids abrupt drops.

Cosine Annealing Formula

$$\eta_t = \eta_{\text{min}} + 0.5 \cdot (\eta_{\text{max}} - \eta_{\text{min}}) \cdot (1 + \cos(\pi \cdot t / T_{\text{max}}))$$

At $t=0$: $\eta_t = \eta_{\text{max}}$ | At $t=T_{\text{max}}$: $\eta_t = \eta_{\text{min}}$ | In between: smooth cosine decay.

```
from torch.optim.lr_scheduler import CosineAnnealingLR
scheduler = CosineAnnealingLR(optimizer, T_max=100, eta_min=1e-6)
```

6.3 Warmup + Decay

Used extensively in transformer training (BERT, GPT, ViT). Start with a very small learning rate, ramp it up linearly for the first K steps (warmup phase), then decay using cosine or linear schedule.

💡 Why Warmup?

At the start of training, weights are random and gradients are large and noisy.

A large initial learning rate causes catastrophic weight updates.

Warmup starts conservatively while the network stabilises, then ramps up to full learning rate once the loss landscape is smoother.

Essential for Transformers; increasingly used for large CNNs as well.

6.4 Cyclical Learning Rates

Instead of monotonically decreasing, cyclical LRs oscillate between a minimum and maximum value. High LRs help escape local minima; low LRs help fine-tune. Cycling between them can outperform either alone.

```
from torch.optim.lr_scheduler import CyclicalLR

scheduler = CyclicalLR(
    optimizer,
    base_lr=1e-4,    # Lower bound
    max_lr=1e-2,     # Upper bound
    step_size_up=2000, # Steps to go from base to max
    mode='triangular' # 'triangular', 'triangular2', 'exp_range'
)
```

Schedule	Pattern	Best For	Key Hyperparameter
Step Decay	Drops by factor every N epochs	CNNs; simple baselines	step_size, gamma
Cosine Annealing	Smooth cosine curve to minimum	Most tasks; default choice	T_max, eta_min
Warmup + Decay	Linear ramp then cosine decay	Transformers; large models	warmup_steps, T_max
Cyclical LR	Oscillates between min/max	Fast convergence; hyperparameter search	base_lr, max_lr, step_size

07 Convolutional Neural Networks (CNN)

Convolutional Neural Networks are the backbone of modern computer vision. Unlike fully-connected networks that treat inputs as flat vectors, CNNs exploit the spatial structure of images through local connectivity and weight sharing — dramatically reducing parameters while capturing hierarchical visual features.

7.1 The Convolution Operation

A convolution slides a small filter (kernel) across the input, computing dot products at each position. This captures local patterns like edges, textures, and shapes, and the same filter is applied everywhere (weight sharing), making CNNs translation-invariant.

2D Convolution Formula

$$(I * K)[i, j] = \sum_m \sum_n I[i+m, j+n] \cdot K[m, n]$$

I = input feature map | K = learnable kernel/filter

Output size = $(H - K_h + 2P) / S + 1$ (for H: input height, K_h : kernel height, P: padding, S: stride)

Key parameters:

Kernel size: typically 3×3 or 5×5

Stride: step size of the filter (stride=2 halves spatial dimensions)

Padding: zeros added to edges to control output size (padding=1 with 3×3 preserves size)

Channels: each conv layer outputs C_{out} channels (one per learned filter)

Why 3×3 Kernels Everywhere?

VGGNet showed that two 3×3 conv layers have the same receptive field as one 5×5 layer, but with fewer parameters ($2 \times 3^2 = 18$ vs $5^2 = 25$) and an extra non-linearity.

Three 3×3 layers match the receptive field of one 7×7 layer with 33% fewer parameters.

Modern architectures (ResNet, EfficientNet) almost exclusively use 3×3 convolutions.

7.2 Pooling Layers

Pooling reduces spatial dimensions, decreasing computation and providing local translation invariance. The most common types are Max Pooling (takes the maximum in each window) and Average Pooling (takes the average).

Pooling Type	Description & Use Case
Max Pooling (most common)	Takes the maximum value in each pooling window. Preserves the strongest feature activation. Standard in CNNs: <code>MaxPool2d(2, 2)</code> halves spatial dimensions.
Average Pooling	Takes the mean value. Smoother downsampling. Used in global pooling (<code>GlobalAvgPool</code>) before classifier — replaces large fully-connected layers in modern nets.

Global Average Pooling	Reduces entire feature map to a single value per channel. Eliminates spatial dimensions entirely. Used in ResNet, EfficientNet, MobileNet before final linear layer.
Adaptive Pooling	Outputs a target size regardless of input size. PyTorch: <code>nn.AdaptiveAvgPool2d((1, 1))</code> → always outputs 1×1 . Essential for variable-size inputs.

7.3 Complete CNN Architecture

A typical CNN follows a pattern of: [Conv → BN → ReLU → Pooling] blocks (feature extraction) followed by fully-connected classifier layers.

```
import torch.nn as nn

class VGGStyleCNN(nn.Module):
    """
    CNN architecture for 32x32 image classification (CIFAR-10 style)
    Follows VGG-style design: progressive channel doubling, 3x3 convs
    """
    def __init__(self, num_classes=10):
        super().__init__()

        self.features = nn.Sequential(
            # Block 1: 32x32x3 → 32x32x64
            nn.Conv2d(3, 64, kernel_size=3, padding=1), # Same padding preserves size
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.Conv2d(64, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2), # 32x32 → 16x16
            nn.Dropout2d(0.25),

            # Block 2: 16x16x64 → 16x16x128
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.Conv2d(128, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2), # 16x16 → 8x8
            nn.Dropout2d(0.25),

            # Block 3: 8x8x128 → 8x8x256
            nn.Conv2d(128, 256, kernel_size=3, padding=1),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2), # 8x8 → 4x4
        )

    # Global Average Pooling instead of large FC layers
```

```
self.global_pool = nn.AdaptiveAvgPool2d((1, 1)) # 4x4x256 → 1x1x256
```

```
self.classifier = nn.Sequential(  
    nn.Dropout(0.5),  
    nn.Linear(256, 128),  
    nn.ReLU(inplace=True),  
    nn.Linear(128, num_classes)  
)
```

```
def forward(self, x):  
    x = self.features(x)  
    x = self.global_pool(x)  
    x = x.flatten(1) # [batch, 256, 1, 1] → [batch, 256]  
    return self.classifier(x)
```

Receptive field grows with depth — key CNN property

Layer 1 (3x3): sees 3x3 region of input

Layer 2 (3x3): sees 5x5 region of input

Layer 3 (3x3): sees 7x7 region of input

7.4 Key CNN Architectures Timeline

Architecture	Year	Key Innovation	Top-1 ImageNet
LeNet-5	1998	First successful CNN; 5 layers; handwritten digit recognition	~99% (MNIST)
AlexNet	2012	Deep CNN on GPU; ReLU; Dropout; started the deep learning revolution	63.3%
VGGNet (VGG-16)	2014	Only 3×3 convolutions; very deep (16/19 layers); simple uniform design	74.5%
GoogLeNet / Inception	2014	Inception modules; parallel filter sizes; 1×1 convolutions for dim reduction	74.8%
ResNet	2015	Residual skip connections; trained networks up to 152 layers	80.6%
EfficientNet	2019	Compound scaling of depth/width/resolution; SOTA efficiency	88.4%
ConvNeXt	2022	CNN architecture modernised to match Vision Transformer performance	87.8%

Residual Skip Connections (ResNet's Key Idea)

Problem: As networks get deeper, gradients vanish during backpropagation.

Solution: Add a shortcut connection that skips 2–3 layers: $y = F(x) + x$

Effect: Gradient can flow directly through the skip connection, bypassing vanishing gradient.

Result: Enabled training of 50, 101, 152-layer networks that actually improved with depth.

Usage: `nn.Sequential` with manual addition of skip — or use `torchvision.models.resnet50()`

08 Object Detection — R-CNN Family

Image classification tells you what is in an image. Object detection tells you what is in an image AND where it is — drawing bounding boxes around each detected object. The R-CNN family represents the evolution of deep learning-based object detection.

8.1 The Object Detection Problem

Object detection requires solving two sub-problems simultaneously: classification (what class is this object?) and localisation (where is the bounding box?). This makes it significantly harder than pure classification.

Task	Description
Image Classification	Input: image → Output: single class label. CNN outputs a softmax vector.
Object Localisation	Input: image → Output: class label + one bounding box (x, y, width, height).
Object Detection	Input: image → Output: multiple class labels + multiple bounding boxes (one per detected object).
Instance Segmentation	Input: image → Output: per-pixel mask for each detected object instance (Mask R-CNN).

8.2 R-CNN (2013) — Region-Based CNN

R-CNN was the first method to successfully apply deep learning to object detection at scale. It uses a two-stage approach: propose regions of interest, then classify each one.

R-CNN Pipeline

Stage 1 — Region Proposals: Use Selective Search to generate ~2,000 region proposals (candidate bounding boxes) from the image. This is a classical computer vision method, NOT learned by the network.

Stage 2 — Feature Extraction: Warp each proposed region to a fixed size (227×227). Pass each region through a CNN (AlexNet) to extract a 4096-dim feature vector.

Stage 3 — Classification: Pass each feature vector to: (a) an SVM classifier for class prediction, (b) a bounding box regressor to refine box coordinates.

RESULT: For each of ~2,000 proposals, you get a class label and refined bounding box.

R-CNN — Pros	R-CNN — Cons
First DL method to outperform classical detection on PASCAL VOC	Extremely slow: ~47 seconds per image (CNN runs 2,000 times per image)
Shows deep features are powerful for detection, not just classification	Huge storage requirement: must cache features for all proposals
Modular: easy to swap CNN backbone	Not end-to-end trainable: CNN, SVM, and regressor trained separately

Landmark paper that established the detection pipeline paradigm

Selective Search is slow and not learned from data

8.3 Fast R-CNN (2015)

Fast R-CNN dramatically improves speed by flipping the order: compute the CNN feature map ONCE for the whole image, then project region proposals onto this shared feature map.

💡 Fast R-CNN Key Innovation — RoI Pooling

Instead of running the CNN separately for each proposal (2,000 CNN passes), run the CNN ONCE on the full image to get a shared feature map.

Project each region proposal onto the shared feature map (just a coordinate lookup — very cheap).

Apply RoI (Region of Interest) Pooling to extract a fixed-size feature vector from each projected region.

RoI Pooling: Divides the projected region into a fixed $H \times W$ grid; applies max pooling in each cell. Output is always $H \times W$ regardless of proposal size.

Pass each RoI feature to a single network head that simultaneously predicts class probabilities AND bounding box offsets.

RESULT: 9× faster than R-CNN at inference. End-to-end training (except region proposals).

```
# Conceptual Fast R-CNN forward pass
import torchvision.ops as ops

# Step 1: Run CNN backbone ONCE on the full image
feature_map = backbone(image) # e.g., [1, 512, 38, 50] for a 600x800 image

# Step 2: Project region proposals onto feature map
# rois: [num_rois, 5] — format: [batch_idx, x1, y1, x2, y2]
rois = get_region_proposals(image) # from Selective Search

# Step 3: RoI Pooling — fixed size output per region
roi_features = ops.roi_pool(
    feature_map, # Shared feature map
    rois, # Region proposals
    output_size=(7, 7), # Fixed output size
    spatial_scale=1/16.0 # Downsampling factor of backbone
) # → [num_rois, 512, 7, 7]

# Step 4: Flatten and pass through detection heads
roi_features = roi_features.flatten(1) # [num_rois, 512*7*7]
class_scores = classifier(roi_features) # [num_rois, num_classes+1]
bbox_offsets = regressor(roi_features) # [num_rois, 4*num_classes]
```

8.4 Faster R-CNN (2015) — End-to-End Detection

The bottleneck of Fast R-CNN is Selective Search for region proposals. Faster R-CNN replaces it with a learned Region Proposal Network (RPN) that shares the backbone CNN — making the entire system end-to-end trainable.

🔍 Region Proposal Network (RPN) — The Key Idea

The RPN slides over the shared CNN feature map (not the original image).

At each spatial location, it predicts k anchor boxes (typically $k=9$: 3 scales $\times$ 3 aspect ratios).

For each anchor: (1) objectness score (object vs background) and (2) bounding box refinement offsets.

Anchors with high objectness scores become region proposals passed to RoI Pooling.

The RPN and detection head share the same backbone CNN — no duplicate computation.

RESULT: 10 $\times$ faster than R-CNN, $\sim$ 5fps. Fully end-to-end trainable with 4-loss joint objective.

```
# Using Faster R-CNN from torchvision (pretrained on COCO, 80 classes)
import torchvision
from torchvision.models.detection import fasterrcnn_resnet50_fpn

# Load pretrained model (ResNet-50 backbone + Feature Pyramid Network)
model = fasterrcnn_resnet50_fpn(pretrained=True)
model.eval()

import torch
from PIL import Image
from torchvision import transforms

# Preprocess image
image = Image.open('image.jpg').convert('RGB')
transform = transforms.ToTensor()
image_tensor = transform(image).unsqueeze(0) # [1, 3, H, W]

# Run inference
with torch.no_grad():
    predictions = model(image_tensor)

# predictions is a list of dicts, one per image:
# predictions[0]['boxes']: [N, 4] — bounding boxes (x1, y1, x2, y2)
# predictions[0]['labels']: [N] — class indices
# predictions[0]['scores']: [N] — confidence scores (0–1)

# Filter by confidence threshold
threshold = 0.5
boxes = predictions[0]['boxes'][predictions[0]['scores'] > threshold]
labels = predictions[0]['labels'][predictions[0]['scores'] > threshold]
scores = predictions[0]['scores'][predictions[0]['scores'] > threshold]

print(f'Detected {len(boxes)} objects with confidence > {threshold}')
```

8.5 Mask R-CNN (2017) — Instance Segmentation

Mask R-CNN extends Faster R-CNN by adding a third head that predicts a per-pixel binary mask for each detected object — enabling instance segmentation (distinguishing individual objects of the same class).

💡 Mask R-CNN Key Innovation — RoI Align

R-CNN and Fast R-CNN use RoI Pooling, which uses integer rounding when projecting proposals onto the feature map. This introduces misalignment that hurts mask prediction.

RoI Align replaces integer rounding with bilinear interpolation, sampling feature values at continuous (non-integer) coordinates.

This small change significantly improves mask accuracy (from ~26% to ~30% mask AP).

Three parallel heads: (1) Classification — class label, (2) Bounding box regression, (3) Mask — 28×28 binary mask per class.

The mask head outputs masks for ALL classes; the predicted class determines which mask to use.

8.6 R-CNN Family Summary

Method	Speed	Key Innovation	Use Case
R-CNN (2013)	~47s/image	Applied deep features to detection for the first time	Historical baseline
Fast R-CNN (2015)	~2.3s/image	Shared CNN + RoI Pooling; end-to-end (mostly)	When proposals are precomputed
Faster R-CNN (2015)	~0.2s/image	Learned RPN replaces Selective Search; fully end-to-end	Two-stage detection standard
Mask R-CNN (2017)	~0.2s/image	Added mask head + RoI Align for instance segmentation	Instance segmentation
YOLO (v1–v8, 2016–)	~0.01s/image	Single-stage: one network, one pass, no proposals	Real-time detection

📄 Two-Stage vs One-Stage Detectors

Two-Stage (Faster R-CNN, Mask R-CNN): Generate proposals, then classify. Higher accuracy, slower.

One-Stage (YOLO, SSD, RetinaNet): Directly predict boxes and classes in one forward pass. Faster, slightly lower accuracy.

For real-time applications (video, robotics): Use YOLO or SSD.

For highest accuracy (medical imaging, satellite): Use Faster R-CNN or Mask R-CNN.

Modern YOLO versions (v8) have closed much of the accuracy gap with two-stage methods.

09 Recurrent Neural Networks (RNN)

While CNNs excel at spatial data (images), Recurrent Neural Networks (RNNs) are designed for sequential data — text, speech, time series, video. They maintain a 'hidden state' that acts as memory, allowing information to persist across time steps.

9.1 The Sequential Data Problem

Standard feedforward networks (MLPs, CNNs) process fixed-size inputs independently. They have no concept of sequence or time. RNNs address this by processing inputs one step at a time while maintaining a hidden state that carries information from previous steps.

Sequential Task	Input → Output Type
Sentiment Analysis	Sequence → One (many-to-one): entire sentence → positive/negative label
Language Generation	One → Sequence (one-to-many): seed word → generated text
Machine Translation	Sequence → Sequence (many-to-many): English sentence → French sentence
Speech Recognition	Sequence → Sequence (many-to-many, different lengths): audio frames → words
Time Series Forecasting	Sequence → One or Sequence: past values → future values

9.2 Vanilla RNN Architecture

A vanilla RNN processes a sequence one element at a time. At each time step t , it takes the current input x_t and the previous hidden state h_{t-1} , combining them to produce a new hidden state h_t .

Vanilla RNN Forward Pass

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$$

$$y_t = W_y \cdot h_t + b_y \quad (\text{output, if needed at each step})$$

h_t = hidden state at time t (the 'memory')

x_t = input at time t

W_h = hidden-to-hidden weight matrix (shared across all time steps)

W_x = input-to-hidden weight matrix (shared across all time steps)

$\tanh$ = activation function (squashes to $[-1, 1]$)

KEY INSIGHT: W_h and W_x are SHARED across all time steps — the same weights process every token. This means the number of parameters does NOT grow with sequence length.

```
import torch.nn as nn
```

```
# Vanilla RNN — simple but suffers from vanishing gradients
rnn = nn.RNN(
    input_size=128,    # Size of each input token embedding
    hidden_size=256,   # Size of hidden state vector
    num_layers=2,      # Stack multiple RNN layers
    batch_first=True,  # Input shape: [batch, seq_len, features]
    dropout=0.3,       # Dropout between layers (not applied to last layer)
    bidirectional=False # If True: processes sequence in both directions
)

# Forward pass
# inputs: [batch_size, seq_len, input_size]
# h0: initial hidden state [num_layers, batch_size, hidden_size]
outputs, hidden = rnn(inputs, h0)
# outputs: [batch_size, seq_len, hidden_size] — output at every step
# hidden: [num_layers, batch_size, hidden_size] — final hidden state
```

9.3 The Vanishing Gradient Problem

Vanilla RNNs suffer from a critical flaw: during backpropagation through time (BPTT), gradients must flow backwards through every time step. Each step involves multiplying by W_h and $\tanh'$. If these multiplications consistently shrink gradients, information from early time steps is lost.

⚠ Vanishing and Exploding Gradients in RNNs

VANISHING: If $|W_h| < 1$, repeated multiplication shrinks gradients exponentially → network forgets long-range dependencies.

EXPLODING: If $|W_h| > 1$, gradients explode → NaN values, training diverges. Fix: gradient clipping.

CONSEQUENCE: Vanilla RNNs effectively have a 'memory' of only ~10–20 time steps.

GRADIENT CLIPPING: Rescale gradients if their norm exceeds a threshold.

`torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)`

BETTER SOLUTION: Use LSTM or GRU cells that mitigate vanishing gradients by design.

9.4 LSTM — Long Short-Term Memory

LSTMs solve the vanishing gradient problem by replacing the simple hidden state with two separate memory pathways: a cell state (long-term memory) and a hidden state (short-term working memory), controlled by three learned gates.

📖 LSTM Equations

$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$ ← Forget Gate: what to ERASE from cell state

$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$ ← Input Gate: what new info to WRITE to cell state

$\tilde{g}_t = \tanh(W_g \cdot [h_{t-1}, x_t] + b_g)$ ← Candidate: candidate values to write

$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$ ← Output Gate: what to READ from cell state

$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{g}_t$ ← Cell state update (long-term memory highway)

$h_t = o_t \odot \tanh(C_t)$ $\leftarrow$ Hidden state output (short-term memory)

$\sigma = \text{sigmoid}$ (output in $[0,1]$) | $\odot$ = element-wise multiplication

Forget gate near 1 $\rightarrow$ keep cell state | Forget gate near 0 $\rightarrow$ erase cell state

💡 Why LSTMs Work — The Cell State Highway

The cell state C_t flows through time with only MULTIPLICATIVE and ADDITIVE interactions (not repeated tanh).

This creates a 'gradient highway' — gradients can flow hundreds of steps with minimal vanishing.

The forget gate learns to preserve important long-range information (e.g., subject-verb agreement across a paragraph).

The input gate controls what new information is written — preventing the cell from being overwritten at every step.

LSTMs can model dependencies of 100s to 1000s of time steps. Vanilla RNNs fail beyond ~20 steps.

```
import torch.nn as nn
import torch

class LSTMTextClassifier(nn.Module):
    """
    LSTM for sequence classification (e.g., sentiment analysis)
    Architecture: Embedding  $\rightarrow$  LSTM  $\rightarrow$  Attention (optional)  $\rightarrow$  Linear  $\rightarrow$  Softmax
    """
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_classes, num_layers=2, dropout=0.3):
        super().__init__()
        # Map integer token IDs to dense vectors
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)

        # LSTM processes the sequence
        self.lstm = nn.LSTM(
            input_size=embed_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=True,
            dropout=dropout,
            bidirectional=True # Process forward AND backward  $\rightarrow$  2x hidden_dim output
        )

        self.dropout = nn.Dropout(dropout)
        # *2 because bidirectional concatenates forward + backward
        self.classifier = nn.Linear(hidden_dim * 2, num_classes)

    def forward(self, token_ids):
        # token_ids: [batch, seq_len]
        x = self.embedding(token_ids) # [batch, seq_len, embed_dim]

        # outputs: [batch, seq_len, hidden_dim*2]
```

```

# hidden: [num_layers*2, batch, hidden_dim] (2x for bidirectional)
outputs, (hidden, cell) = self.lstm(x)

# Use the last time step output for classification
# Or: use the mean of all outputs for better coverage
context = outputs[:, -1, :] # [batch, hidden_dim*2]
context = self.dropout(context)
return self.classifier(context) # [batch, num_classes]

# Example usage
model = LSTMTextClassifier(
    vocab_size=30000, embed_dim=256, hidden_dim=512,
    num_classes=2, num_layers=2, dropout=0.3
)

```

9.5 GRU — Gated Recurrent Unit

GRU is a simplified version of LSTM proposed by Cho et al. (2014). It merges the cell state and hidden state into a single vector and uses only two gates instead of three — making it faster to train with comparable performance on many tasks.

GRU Equations

$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$ ← Update Gate: balance between old and new hidden state
 $r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$ ← Reset Gate: how much past to forget when computing candidate
 $\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t])$ ← Candidate hidden state
 $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$

Update gate near 1 → mostly use new candidate (fast updating)

Update gate near 0 → mostly keep previous hidden state (long-term memory)

Property	LSTM vs GRU
Number of gates	LSTM: 3 (forget, input, output) vs GRU: 2 (update, reset)
Memory	LSTM: separate cell state + hidden state vs GRU: single hidden state
Parameters	LSTM: ~4× more parameters than vanilla RNN vs GRU: ~3× (fewer than LSTM)
Training speed	GRU is typically 20-30% faster to train than LSTM
Performance	Comparable in practice; LSTM slightly better for very long sequences
When to use	LSTM: very long sequences, complex dependencies GRU: shorter sequences, speed matters

```
# GRU in PyTorch — same API as LSTM, no cell state
gru = nn.GRU(
    input_size=128,
    hidden_size=256,
    num_layers=2,
    batch_first=True,
    dropout=0.3,
    bidirectional=True
)

# Forward pass — NOTE: no cell state output (unlike LSTM)
outputs, hidden = gru(inputs) # No (hidden, cell) tuple
# outputs: [batch, seq_len, hidden_size * 2] (bidirectional)
# hidden: [num_layers * 2, batch, hidden_size]
```

9.6 Bidirectional RNNs

In many tasks, context from BOTH the past AND the future helps. Bidirectional RNNs process the sequence twice — once forward and once backward — and concatenate the hidden states, giving each position access to its entire context.

💡 When to Use Bidirectional RNNs

USE bidirectional when: entire sequence is available at inference time (text classification, NER, translation encoder).

DON'T use bidirectional when: generating sequences token-by-token (language generation, real-time speech), because future tokens aren't available.

BERT uses bidirectional attention (not RNN), which is why it excels at understanding tasks but can't generate text autoregressively.

Output dimension doubles: hidden_dim=256 bidirectional → 512-dim output (forward 256 + backward 256 concatenated).

9.7 Sequence-to-Sequence (Seq2Seq) with Encoder-Decoder

Many real-world tasks (translation, summarisation, question answering) require mapping a variable-length input sequence to a variable-length output sequence. The Encoder-Decoder architecture solves this.

```
class Seq2SeqEncoder(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_layers=2, dropout=0.3):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, num_layers,
                             batch_first=True, dropout=dropout)

    def forward(self, src):
        # src: [batch, src_len]
        embedded = self.embedding(src) # [batch, src_len, embed_dim]
        outputs, (hidden, cell) = self.lstm(embedded)
```

```
# outputs: all encoder hidden states (needed for attention)
# hidden/cell: final state — becomes initial state of decoder
return outputs, hidden, cell
```

```
class Seq2SeqDecoder(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_layers=2, dropout=0.3):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, num_layers,
                             batch_first=True, dropout=dropout)
        self.output_proj = nn.Linear(hidden_dim, vocab_size)

    def forward(self, tgt_token, hidden, cell):
        # tgt_token: [batch] — one token at a time (during inference)
        tgt_token = tgt_token.unsqueeze(1) # [batch, 1]
        embedded = self.embedding(tgt_token) # [batch, 1, embed_dim]
        output, (hidden, cell) = self.lstm(embedded, (hidden, cell))
        prediction = self.output_proj(output.squeeze(1)) # [batch, vocab_size]
        return prediction, hidden, cell
```

9.8 RNN vs Transformer — When to Use Each

Criterion	RNN (LSTM/GRU) vs Transformer
Sequential computation	RNNs must process tokens in order (cannot parallelise sequence processing during training). Transformers process all tokens simultaneously.
Training speed	RNNs are slow to train on long sequences due to sequential dependency. Transformers train much faster with full parallelism.
Long-range dependencies	LSTMs handle ~100–500 token dependencies. Transformers (with attention) handle full sequence length with direct connections.
Real-time streaming	RNNs update state one token at a time — ideal for streaming input (speech, sensor data). Transformers need full context.
Parameter efficiency	RNNs: $O(n)$ parameters (shared weights). Transformers: $O(n^2)$ attention — expensive for very long sequences.
Current dominance	Transformers dominate NLP (BERT, GPT). RNNs still used in time series, streaming, and edge deployment.

10 Hands-On Lab — CIFAR-10 CNN with PyTorch

Hands-On Lab Objective

Train a Convolutional Neural Network on CIFAR-10 applying all Day 8 techniques:

- AdamW optimiser with decoupled weight decay
- Dropout (both spatial Dropout2d and standard Dropout)
- Batch Normalisation after every conv layer
- Cosine Annealing learning rate scheduler
- Early stopping with best-model checkpointing

Target: >75% test accuracy on CIFAR-10 (10-class colour image classification, 60,000 images, 32×32 pixels)

CIFAR-10 Classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim.lr_scheduler import CosineAnnealingLR
import torchvision
import torchvision.transforms as transforms

# — Data Loading & Augmentation —————
transform_train = transforms.Compose([
    transforms.RandomHorizontalFlip(),      # 50% chance horizontal flip
    transforms.RandomCrop(32, padding=4),   # Random crop with 4px padding
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2), # Color augment
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2470, 0.2435, 0.2616))
])

transform_test = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2470, 0.2435, 0.2616))
])

train_set = torchvision.datasets.CIFAR10(root='./data', train=True, download=True, transform=transform_train)
test_set = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, transform=transform_test)
train_loader = torch.utils.data.DataLoader(train_set, batch_size=128, shuffle=True, num_workers=2)
test_loader = torch.utils.data.DataLoader(test_set, batch_size=128, shuffle=False, num_workers=2)

# — Model Definition (CNN + BN + Dropout) —————
class CIFAR10CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            # Block 1: 32x32x3 → 16x16x64
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32), nn.ReLU(inplace=True),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64), nn.ReLU(inplace=True),
```

```

        nn.MaxPool2d(2, 2),
        nn.Dropout2d(0.25),

        # Block 2: 16x16x64 → 8x8x128
        nn.Conv2d(64, 128, kernel_size=3, padding=1),
        nn.BatchNorm2d(128), nn.ReLU(inplace=True),
        nn.Conv2d(128, 128, kernel_size=3, padding=1),
        nn.BatchNorm2d(128), nn.ReLU(inplace=True),
        nn.MaxPool2d(2, 2),
        nn.Dropout2d(0.25),
    )

    self.classifier = nn.Sequential(
        nn.Linear(128 * 8 * 8, 512),
        nn.ReLU(inplace=True),
        nn.Dropout(0.5),
        nn.Linear(512, 10)
    )

    def forward(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)
        return self.classifier(x)

# — Training Setup —————
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = CIFAR10CNN().to(device)
optimizer = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
scheduler = CosineAnnealingLR(optimizer, T_max=100, eta_min=1e-6)
criterion = nn.CrossEntropyLoss()

# — Training Loop —————
def train_epoch(model, loader, optimizer, criterion, device):
    model.train()
    total_loss, correct = 0.0, 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * images.size(0)
        correct += (outputs.argmax(1) == labels).sum().item()
    return total_loss / len(loader.dataset), correct / len(loader.dataset)

def evaluate(model, loader, criterion, device):
    model.eval() # ← Critical: disables Dropout, uses BN running stats
    total_loss, correct = 0.0, 0
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)

```

```

        total_loss += loss.item() * images.size(0)
        correct += (outputs.argmax(1) == labels).sum().item()
    return total_loss / len(loader.dataset), correct / len(loader.dataset)

# — Run Training with Early Stopping —————
best_val_acc = 0.0
patience, wait = 10, 0

for epoch in range(100):
    train_loss, train_acc = train_epoch(model, train_loader, optimizer, criterion, device)
    val_loss, val_acc = evaluate(model, test_loader, criterion, device)
    scheduler.step()

    current_lr = scheduler.get_last_lr()[0]
    print(f'Epoch {epoch+1:3d} | LR: {current_lr:.6f} | Train Loss: {train_loss:.4f} Acc: {train_acc:.3f} | Val Loss: {val_loss:.4f} Acc: {val_acc:.3f}')

    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save(model.state_dict(), 'best_model.pt')
        wait = 0
    else:
        wait += 1
        if wait >= patience:
            print(f'Early stopping at epoch {epoch+1}. Best Val Acc: {best_val_acc:.3f}')
            break

print(f'Training complete. Best Validation Accuracy: {best_val_acc:.4f}')

```

10.1 Ablation Study

Experiment	Change from Baseline	Expected Accuracy	Primary Learning
Baseline	No BN, No Dropout, Fixed LR (1e-3), SGD	~60–65%	Understand impact of each technique
+ BatchNorm	Add BN to all conv layers	~65–73%	BN effect on convergence stability
+ Dropout	Add Dropout2d(0.25) + Dropout(0.5)	~68–76%	Regularisation impact on overfitting
+ Adam → AdamW	Switch from SGD to AdamW	~70–78%	Optimiser selection impact
+ Cosine LR	Add CosineAnnealingLR	~72–80%	LR scheduling benefit
+ Data Augmentation	Add ColorJitter, RandomCrop	~75–83%	Augmentation as regularisation
Full (all combined)	All techniques together	~78–85%	Synergistic effect of all techniques

11 Glossary & Review Questions

11.1 Key Terms

Term	Definition
Epoch	One complete pass through the entire training dataset.
Batch / Mini-batch	A subset of samples processed together before a weight update.
Iteration / Step	One forward + backward pass + weight update on one batch.
Overfitting	Model memorises training data; high train accuracy, low val accuracy.
Underfitting	Model too simple; poor performance on both training and validation sets.
Bias	Error from overly simplistic assumptions in the model (too simple).
Variance	Sensitivity to fluctuations in the training set (too complex / overfits).
Bias-Variance Tradeoff	The fundamental tension between model simplicity (high bias) and flexibility (high variance).
L1 Regularisation	Penalty proportional to absolute weight values; promotes sparsity (many weights = 0).
L2 Regularisation	Penalty proportional to squared weight values; promotes small weights.
Dropout	Randomly zeros neurons during training to prevent co-adaptation; ensemble effect.
Early Stopping	Halt training when validation loss stops improving to prevent overfitting.
Momentum	Accumulates gradient history to accelerate optimisation in consistent directions.
Adam	Adaptive optimiser combining momentum + per-parameter learning rates.
AdamW	Adam with decoupled weight decay; preferred for transformers and large models.
Symmetry Problem	Zero-init causes all neurons to learn identically; no differentiation possible.
Xavier Init	Weight init scaled by $\sqrt{6/(\text{fan_in} + \text{fan_out})}$; designed for Sigmoid/Tanh.
He Init	Weight init scaled by $\sqrt{2/\text{fan_in}}$; designed for ReLU networks.
Internal Covariate Shift	Changing distribution of layer inputs as weights are updated during training.
Batch Normalisation	Normalises layer inputs to zero mean and unit variance per mini-batch.
Running Statistics	Moving averages of μ and σ accumulated during training; used at inference time.
Step Decay	Multiply LR by γ every N epochs — simple but requires manual tuning.
Cosine Annealing	Decay LR following a cosine curve from max to min over T_{max} epochs.

Warmup	Gradually increase LR from near-zero at the start of training to full learning rate.
Cyclical LR	Oscillate LR between min and max on a repeating schedule to escape local minima.
Convolution	Sliding dot product of a learned filter over the input; detects local patterns.
Receptive Field	The region of the input image that a neuron in a given layer 'sees'.
RoI Pooling / RoI Align	Extracts fixed-size features from variable-size region proposals (detection models).
Region Proposal Network	Learned network in Faster R-CNN that predicts candidate object locations.
Anchor Boxes	Predefined bounding boxes of different sizes/ratios used as detection references in RPN.
RNN Hidden State	Compressed representation of sequence history; passed from one time step to the next.
LSTM Cell State	Separate long-term memory pathway in LSTM that mitigates vanishing gradients.
Vanishing Gradient	Gradients shrink to near-zero during BPTT; early time steps are forgotten.
Bidirectional RNN	Processes sequence forward AND backward; combines both hidden states per position.
Seq2Seq	Encoder-Decoder architecture for variable-length input to variable-length output tasks.
Diffusion Model	Generative model that learns to reverse a noise-adding process to create images.
CFG (Classifier-Free Guidance)	Guidance scale controlling how strongly a diffusion model follows the text prompt.

11.2 Review Questions

Answer the following questions to test your understanding of Day 8 material.

Original Day 8 Conceptual Questions

Q1: You train a model for 50 epochs. Training loss reaches 0.05 but validation loss is 0.45 and rising. What is happening, and what are three concrete steps you would take to address it?

Q2: Explain the symmetry problem in weight initialisation. Why does zero initialisation fail even though small random initialisation works?

Q3: Your colleague trains a model using Adam with L2 regularisation (adding a penalty term to the loss). You suggest switching to AdamW. What is the technical difference, and why does it matter?

Q4: A model uses Batch Normalisation. During deployment, predictions are inconsistent and much worse than validation performance. What is the most likely cause, and how do you fix it?

Q5: Compare Step Decay and Cosine Annealing scheduling. When would you prefer one over the other?

Original Day 8 Practical Questions

Q6: Write the PyTorch code to add Dropout ($p=0.5$) between two linear layers, and show the code required to correctly switch between training and evaluation modes.

Q7: Implement He initialisation manually for a `nn.Linear` layer in PyTorch.

Q8: Given the CIFAR-10 training script from the lab, modify it to use SGD + Momentum instead of AdamW. What hyperparameters need to change?

Original Day 8 Challenge Questions

Q9: The Bias-Variance tradeoff suggests you cannot simultaneously reduce both. How does Dropout reduce variance without substantially increasing bias? Use the ensemble interpretation.

Q10: Why does Batch Normalisation act as a regulariser? What is the source of the regularisation effect, and in what circumstances might it be harmful?

Extended Questions (CNN, R-CNN, RNN, Image Generation)

Q11: Write PyTorch code for a Bidirectional LSTM text classifier with 2 layers, hidden size 256, dropout 0.3.

Q12: Implement a complete Faster R-CNN inference pipeline in PyTorch using torchvision. Filter detections by confidence threshold.

Q13: Modify the CIFAR-10 training script to use SGD + Momentum (0.9) + Cosine Annealing instead of AdamW. What hyperparameters change?

Q14: Write an effective image generation prompt for a visualisation of gradient descent in a 3D loss landscape.

Q15: Compare GRU and LSTM. In what tasks and sequence lengths would you expect each to have an advantage?

Q16: You are building a real-time pedestrian detection system for a self-driving car at 30fps. Compare Faster R-CNN vs YOLOv8. What are the key tradeoffs?

The future of AI is bright, and you can be part of it!